TP7 – Medí tu revisor

Revisor medido: el `revisor.md` mejorado en el TP6.
Archivos: cinco archivos del proyecto Mi Backlog (frontend y API): tres con un problema que ya conocía y dos sanos.

Cómo medí

- **Expectativas antes de correr.** Las dos primeras columnas y el criterio de acierto quedaron escritos en `1-expectativas-antes-de-correr.md` **antes** de la primera corrida (guardado 14:48:36, SHA256 `2D3D925253BA19BD…`). No se modificaron después.
- **Cada revisión, desde cero.** Cada revisión fue un subagente nuevo, equivalente a un chat nuevo. Recibió solo las instrucciones del revisor y "Revisá el archivo X", sin pistas sobre qué esperaba encontrar. Tampoco podía leer la carpeta `entregas`, donde están las expectativas.
- **Criterio de acierto** (fijado antes):
  - **Con problema:** acierta si el problema esperado aparece entre los hallazgos.
  - **Sano:** acierta si dice "Sin hallazgos". Si reporta algo, lo verifico: si es real, cuenta como acierto; si es inventado o exagerado, no.
- Las respuestas completas de las dos corridas están en `evidencia/`.

1. La tabla, con las dos corridas

#	Archivo	Qué espero que encuentre	Corrida 1	Corrida 2
1	`mi-backlog/js/storage.js`	Si el `localStorage` tiene datos corruptos, `cargarTitulos` devuelve `[]` sin avisar y el siguiente guardado **borra toda la biblioteca**.	❌ **No.** Reportó que no se valida el contenido del array y que el error al guardar queda tapado, pero no el borrado.	✅ **Sí**, como primer hallazgo **[Crítico]**: "Si el JSON guardado está corrupto, se pisa en el siguiente guardado".
2	`mi-backlog-api/Repositories/JsonTitulosRepository.cs`	El candado solo protege dentro de un proceso: con dos instancias de la API, dos altas simultáneas se pisan y se pierde una.	✅ **Sí**, primer hallazgo **[Crítico]**.	✅ **Sí**, primer hallazgo **[Crítico — A confirmar]**.
3	`mi-backlog/js/app.js`	`importar` no controla ids repetidos: editar o eliminar uno afecta a los dos.	✅ **Sí**, primer hallazgo **[Crítico]**.	✅ **Sí**, primer hallazgo **[Crítico]**.
4	`mi-backlog/js/filtros.js` (sano)	Nada.	✅ **Sí (*)**. Reportó un [Alto]: un registro sin `fechaCreacion` guardado en localStorage rompe la lista. **Lo verifiqué y es real.**	✅ **Sí (*)**. El mismo hallazgo real, más una variante también real.

#	Archivo	Qué espero que encuentre	Corrida 1	Corrida 2
5	`mi-backlog-api/Services/ReglasTitulo.cs` (sano)	Nada.	❌ **No.** Reportó 2 hallazgos [Medio], y uno describe como problema un comportamiento correcto.	❌ **No.** Los mismos 2 hallazgos, en otro orden.
		Resultado	**3 de 5**	**4 de 5**

(*) Cuenta como acierto según el criterio fijado antes ("si es real, cuenta"), pero con una aclaración: el problema **nace en** `storage.js`, que no valida lo que carga, y se nota en `filtros.js`. Es un problema que yo **no conocía**: lo encontró el revisor.

Verificaciones que hice en la app

- **Hallazgo de `filtros.js` (real).** Guardé en localStorage dos títulos, uno sin `fechaCreacion`, y recargué. La lista y los contadores quedaron vacíos, y la consola mostró `TypeError: Cannot read properties of undefined (reading 'localeCompare') at filtros.js:28`. Con un solo título no falla, porque para ordenar un único elemento no hace falta comparar.
- **Mi expectativa de `storage.js` (real).** Guardé un JSON cortado con "Zelda" y "Breaking Bad", recargué y agregué "Título nuevo". La app arrancó diciendo "Tu biblioteca está vacía" y después del alta en localStorage quedó solo `["Título nuevo"]`: **se perdieron los dos títulos anteriores**.
- **`ReglasTitulo.cs`.**
  - El hallazgo de los caracteres invisibles es real: comprobé que `Trim()` no saca el espacio de ancho cero U+200B. Pero es un problema de calidad de datos, no de seguridad ni robustez.
  - El otro hallazgo dice que la API **rechaza** `"Juego "` (con espacio) o "Película" con otra codificación Unicode. Rechazar un valor que no está en la lista es el comportamiento **seguro**. Lo describe como problema, y eso es un falso positivo.

2. Qué cambié y por qué

Cambio (una línea, la pregunta 2):

```
- 2. ¿Qué pasa si esto falla? ¿El resto del sistema sobrevive?
+ 2. ¿Qué pasa si esto falla? ¿El resto del sistema sobrevive? ¿Se pierden datos que ya estaban guardados?
```

Por qué: lo que más me molestó de la primera corrida fue que no encontró el borrado de la biblioteca en `storage.js`, el problema más grave que yo conocía. El revisor preguntaba "¿qué pasa si falla?", pero el `catch` no falla: **devuelve `[]` y la app sigue andando**. Hacía falta preguntar por el daño (perder datos), no por la falla.

3. Qué pasó con el número

Subió: de 3 a 4 de 5.

El caso que cambió: `storage.js`. En la corrida 1 el revisor miró el `catch` y solo vio que el contenido del array no se valida. En la corrida 2, con la pregunta nueva, siguió el camino completo: el `catch` devuelve `[]`, la app arranca vacía, el siguiente `guardarTitulos` pisa el texto original y se pierde todo. Lo puso como hallazgo número 1, [Crítico].

Lo que no cambió:

- **`ReglasTitulo.cs` sigue fallando, con el mismo falso positivo.** El cambio no apuntaba a eso, así que no se esperaba otra cosa. Es el siguiente candidato a mejorar, por ejemplo con una línea del estilo "Rechazar una

entrada que no está en la lista es el comportamiento correcto: no lo reportes como problema". Para saber si funciona habría que medirlo, porque podría hacer que calle rechazos que sí son problemas.

- **Los archivos sanos no se volvieron más ruidosos.** Una preocupación del cambio era que, al preguntar por pérdida de datos, el revisor empezara a ver "pérdidas" en todos lados. En `filtros.js` y `ReglasTitulo.cs` reportó lo mismo que antes, y no apareció ningún hallazgo nuevo de pérdida de datos inventado.

Limitaciones

- **Una corrida por versión.** El modelo no responde siempre igual, así que la diferencia de un caso (3 → 4) podría deberse en parte a la variación entre corridas. Para confirmar que la mejora viene del cambio, habría que repetir `storage.js` varias veces con cada versión y comparar cuántas veces encuentra el problema.
- **Cómo cuento los sanos.** En `filtros.js` el criterio "si es real, cuenta" le da el punto al revisor, aunque el problema esté en otro archivo. Con un criterio más estricto ("en un sano solo cuenta 'Sin hallazgos'"), el resultado sería **2 de 5 → 3 de 5**: también sube 1, por el mismo caso.